RoboGPU: Accelerating GPU Collision Detection for Robotics

Lufei Liu, Liwei Xue, Yuan Hsi Chou, Jocelyn Zhao, Lara Kawasme, Youssef Mohammed, Tor M. Aamodt
University of British Columbia
Vancouver, Canada
liulufei@student.ubc.ca, aamodt@ece.ubc.ca

***Abstract*—Autonomous robots are anticipated to be deployed soon in domains ranging from transportation to healthcare and home assistance. Enabling autonomous robotics requires a computation platform flexible enough to execute a diverse and evolving collection of workloads while meeting real-time requirements. We believe a GPU-like architecture will be a key component of such platforms. Recent GPUs combine a flexible parallel processing fabric augmented with efficient support for important application domains via embedded accelerators (e.g., Tensor Cores), and a GPU-like architecture has reportedly been adopted for the Tesla AI5 accelerator. While current GPUs are effective at supporting emerging neural motion planners, we find that collision detection is crucial for evaluating their proposed trajectories and that this step appears to require dedicated acceleration to operate in real-time. In this work, we propose RoboCore, an accelerator block embedded within a robotics-focused GPU (RoboGPU) architecture. We explore and compare architectural modifications to address the gaps of existing ray tracing accelerators (RTAs) for robotics and find that RoboCore computes collision queries 2.8× faster than RTA implementations using 48% less energy with 2% more area than RTAs. RoboCore is 13.3× faster than a CUDA baseline, and achieves 3.4× end-to-end speedup on a neural motion planner and 1.1× speedup on Monte Carlo Localization compared to a baseline GPU. This demonstrates that a hybrid approach of embedded specialization within a flexible general-purpose GPU architecture is suitable for supporting advancements in robotics.**

***Index Terms*—GPU, Collision Detection, Robotics, Motion Planning**

I. Introduction

Autonomous robots have significant potential across industries, including self-driving vehicles, healthcare, and home assistance. Recent advances in neural motion planning, both in path planning with MPNet [1] and MπNet [2] and vision-language-action (VLA) models such as NVIDIA's Alpamayo-R1 [3] have brought the development of truly autonomous robots closer to reality. These models help robots interpret environments, identify goals, and compute feasible trajectories to follow. Companies such as Tesla, Waymo, and Mercedes are already shifting from classical algorithms to neural models in self-driving vehicles and humanoid robots [4]–[7], and we speculate that this trend will continue as more robots make the transition. However, despite a growing number of startups, some roboticists believe we may still be a decade from widespread robotics deployment in everyday life [8]. Thus, robotics algorithms will likely continue to evolve and consequently need a flexible yet efficient compute platforms to handle the real-time compute-intensive tasks robotics requires.

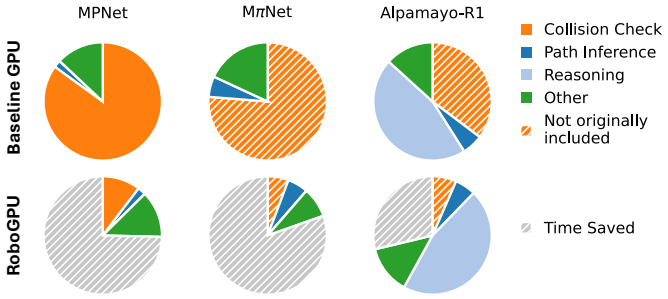


Fig. 1. Latency distribution of neural planning on MPNet, MπNet, and Alpamayo-R1 with explicit collision checking (appended for MπNet and Alpamayo-R1).

Moreover, despite recent success rate improvements of published neural motion planners, empirical results demonstrate these models remain imperfect. Thus, similar to Qureshi et al. [1], we assume that future systems deploying neural planners autonomously in safety critical environments where actual collisions are unacceptable will couple neural path planning with explicit collision detection [2], [9]. This is akin to the philosophy underlying DIVA [10], in which a validated in-order checker-processor can quickly verify the correctness of already executed instructions that are about to be committed by a fast, possibly buggy, out-of-order core since dependencies are decoupled post-execution. Since verifying the behavior of neural networks is challenging [11], we study a system in which a path is found using an approximate neural planner (not proven to guarantee safety) and the proposed path is checked using a more conservative classical collision detection algorithm before control outputs would be sent to physical actuators.

While recently proposed neural planners are trained with collision-avoidance objectives, they are known to still produce paths that collide with obstacles in the environment. For example, MπNet reports 99% success in finding a path reaching the target goal but the proposed paths result in a 14% collision rate, bringing the actual collision-free success rate to just 80.80% [9]. AvoidEverything [9] results in a 3.10% collision rate despite optimizing specifically for collision avoidance leading to a 92.34% success rate. More recently, Alpamayo-

R1 [3] reports a 9% close encounter rate for their 0.5B real-time model, which is non-negligible and likely beyond reasonable incidents per million miles (IPMM) metrics for self-driving vehicles [12]. Avoiding collisions using neural models becomes more difficult for humanoid robots with higher degrees of freedom [13] that cannot easily recover from instability or falling. Dedicated collision avoidance units exist in many robots already [6], [14], [15]. Therefore, consistent with limitations identified in the neural path planning literature [2], [9], we believe explicit collision detection will be crucial in the robotics pipeline when adopting neural motion planners for safety-critical tasks.

However, collision detection is known to be costly both in time and energy [16]–[18]. Figure 1 shows the latency distribution of neural planning with MPNet and MπNet measured on a NVIDIA RTX 5070 GPU, and Alpamayo-R1 using reported results (top). MPNet already includes collision checking and we add collision checking to MπNet and Alpamayo-R1 to estimate the latency impact. Although collision detection is a smaller component in Alpamayo-R1 where reasoning latency dominates, reasoning only produces the target goal, while path planning and collision checking are repeatedly executed for replanning. Collision queries add an additional 76% latency to low level planners like MπNet and remain a bottleneck at 65% of replanning latency for a VLA model. Figure 1 (bottom) shows how a dedicated robotics architecture, RoboGPU, can reduce the overall latency while ensuring safe collision checking, ultimately resulting in improved robotic responsiveness [19].

Many specialized accelerators have been proposed to meet the latency requirements of robotics tasks [20]–[24], but they are limited to specific algorithms and cannot adapt to new robotics advancements. The number of papers accepted at ICRA, the leading robotics conference, has roughly doubled in the past ten years [25], showing robotics algorithms are evolving rapidly. To leverage the benefits, there is a need to combine programmability with efficiency in robotics computation platforms. We speculate that system-on-chips integrating general-purpose Graphics Processing Units (GPUs) are strong candidates for robotics acceleration due to their existing combination of programmability and high computational performance. For example, NVIDIA's Jetson GPUs are designed with the flexibility to target robotics applications across a range of domains and Tesla recently indicated AI5, their latest chip, is a GPU-like architecture optimized for autonomous vehicles[1].

Today's GPUs increasingly incorporate hardened accelerator blocks such as Tensor Cores, Tensor Memory Accelerators, and Ray Tracing Cores to improve performance and efficiency without sacrificing flexibility, similar to how embedding "hard" multiplier and memory blocks were adopted within FPGA fabrics two decades ago to help close area, performance, and power gaps between completely flexible lookup tables and fixed function ASICs [27]. Despite strong competition [28], NVIDIA has repeatedly shown that the tension between programmability and efficiency can be balanced in part by integrating such hardware units into the general-purpose GPU architecture united by the CUDA programming model. This approach to balancing efficiency and flexibility is also consistent with earlier academic studies [29]. We speculate that with adoption of neural techniques, adding such a dedicated robotics accelerator block for collision detection will be increasingly helpful for ensuring safety, especially as software collision detection struggles to meet real-time latency constraints. Figure 2 (top left) highlights this issue appearing as low SIMT efficiency (threads active per warp due to control flow) for CUDA collision queries on a NVIDIA RTX 2080 Ti GPU, causing underutilization.

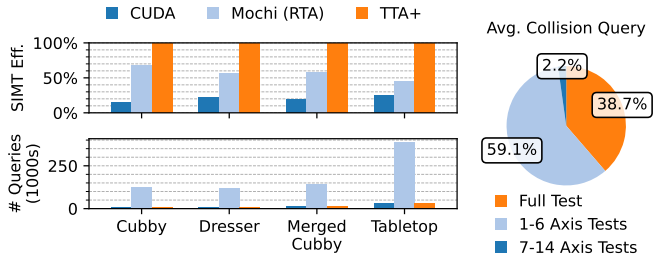


Fig. 2. SIMT efficiency and collision queries required by each device on environments from MπNet [2] (left). Required computation for the average collision query (right).

We explore collision detection using a dedicated hardware unit *within* the GPU to achieve real-time performance, as offloading queries to the CPU or other accelerators would incur severe communication overheads and latency, especially considering recent trends of CPU orchestration bottlenecks [30]. A close match to the hardware requirements is the Ray Tracing Accelerator (RTA) in current GPUs [31] since the ray tracing operation is also divergent and generates irregular memory accesses in 3D space, with several works already attempting to leverage RTAs for robotics workloads [32]–[34]. Figure 2 shows that collision queries on the RTA using Mochi [32] hides the divergence better than CUDA but requires 12$\times$ more queries in its software-only implementation for the same task due to the RTA interface. Other generalized versions of RTAs, such as TTA+ [35], can achieve higher SIMT efficiency without the overheads of regular RTAs but suffer different inefficiencies for collision queries. Although RTAs are not ideal for robotics, using RTAs as the foundation for RoboCore allows for minimal design costs and leverage of existing hardware and software infrastructure.

In this paper, we propose RoboCore, a hardware unit that integrates into existing GPU architectures to create RoboGPU and provide an RTA-like platform optimized for robotics workloads. We focus our design on accelerating explicit collision detection as a step towards safe deployment of neural motion planners in real-world robotics. By placing RoboCore within

[1]Elon Musk, describing deletion of the "legacy GPU" in the upcoming AI5 self-driving chip, suggested this was motivated "because [AI5] basically *is* a GPU" for which "excess production can always be put in our data centers [alongside NVIDIA GPUs]." [26]

the GPU, we can eliminate data transfers from the neural planner to RoboCore for collision queries, tightly coupling the two stages for efficient planning iterations. We ensure RoboCore also adapts well to other important kernels in the robotics pipeline such as point cloud processing and classical robotics algorithms such as Monte Carlo Localization, highlighting its flexibility to support advancements in robotics. Our contributions are as follows:

- We propose RoboGPU, a robotics-focused GPU that integrates our RoboCore design to enable fast collision detection queries, achieving 2.8× speedup over existing RTA implementations while maintaining programmability.
- We compare design choices for RoboCore and analyze their impact on performance and efficiency for robotics collision detection, resulting in 48% energy savings with 2% area overhead over RTAs.
- We demonstrate that RoboCore extends beyond collision detection to a subset of robotics kernels, accelerating point cloud processing in MπNet [2] by 3.4× and Monte Carlo Localization from the RoWild [36] benchmark by 1.1× over a baseline GPU.

II. Background

This section provides an overview of motion planning for robotics, including neural motion planners and collision detection. We also describe details of RTAs in modern GPUs and prior works that leverage RTAs for alternative workloads.

A. Motion Planning for Robotics

Motion planning in robotics aims to identify a feasible trajectory for a robot to move from its current position to reach a target goal, broken into a sequence of discretized milestones. Traditionally, these milestones are generated using algorithmic methods, including search-based planning algorithms [37], [38] and sampling-based algorithms [39]–[41]. With the rise of deep learning, neural path planners such as MπNet [2] and others [1], [42]–[44] have been introduced, seeking to imitate efficient samplers to speed up traditional planners. These planners all require a meaningful representation of the environment, typically derived from sensor inputs such as LiDAR or RGB-D cameras that are preprocessed by other neural networks like PointNet++ [45] to extract relevant features. More recently, vision-language-action (VLA) models such as HDP [46] and Alpamayo-R1 [3] have combined these tasks into a single end-to-end model that takes sensor inputs and natural language instructions to directly output a robot path. However, the underlying architecture of these VLA models still generally consists of an encoder to understand the environment, a reasoning module to identify the target goal, and a low-level path planner that generates trajectory milestones, summarized with latencies in Figure 3. Low-level path planners are often implemented with diffusion models, inspired by Diffusion Policy [47], which are used for both Alpamayo-R1 and HDP. Although neural motion planning networks are becoming increasingly powerful and attaining higher success rates, the failure rate increases significantly with the complexity of the environment and the difficulty of the end goal.

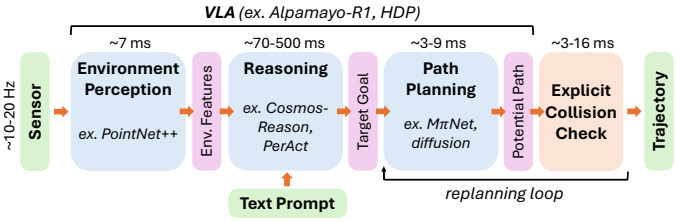


Fig. 3. Robotics pipeline with VLA model and collision checking. The VLA model consists of individual stages with sensors and text as inputs and a potential path as the output.

B. Collision Detection

Collision detection identifies whether a robot will collide with obstacles in its environment along a path, and is crucial for safety when using neural planners [2], [9]. Although there are neural models for collision detection like DeepCollide [13], we focus on classical algorithms to avoid introducing additional errors. Collision checks can be tightly coupled with the path planning stage in Figure 3 to select between potential paths and replan when necessary. The replanning loop is especially time-sensitive as the robot must react to environmental changes and update its path in real-time, whereas the initial goal reasoning stage can be more flexible in latency.

To perform collision checks, the potential robot path is discretized and the robot geometry at each step can be represented by oriented bounding boxes (OBBs) for a conservative estimate. The environment can be represented with axis-aligned bounding boxes (AABBs). AABBs, unlike OBBs, are aligned with the coordinate axes, making them simpler to compute and store. These AABBs are then stored in spatial data structures, such as the octree that recursively subdivides the 3D space into octants hierarchically, or the bounding volume hierarchies (BVH) that tightly bounds groups of nearby AABBs (possibly overlapping) into larger AABBs to form a tree structure. We evaluate octrees over BVHs since they are easier to store and update, which is more suitable for robotics, although the AABBs used for the collision algorithm are the same for both. Octree construction and updates are a time consuming process and can potentially impact the overall performance of collision detection. However, we focus on accelerating the collision query stage in this work and rely on existing octree optimization strategies [33], [48]–[50] to mitigate this issue.

Collision detection between the robot and environment is performed by traversing the octree and checking for intersections between the robot OBBs and the occupied environment AABBs at each step. Figure 4 illustrates the separating axis collision test (SACT) used for OBB-AABB intersection testing [51]. Both the AABB and OBB are projected onto potential separating axes, and if there exists a separating line on any axis that divides the projections of the two boxes, then the boxes are not colliding. The SACT creates many

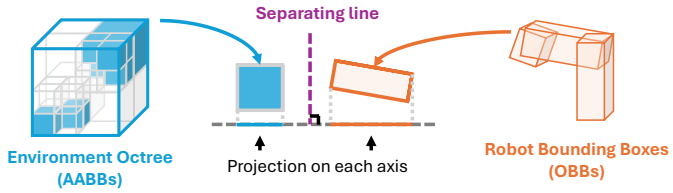


Fig. 4. Separating axis collision test (SACT) between AABB from environment octree and OBB from robot. If a separating line divides the projections of the two boxes, then the boxes are not colliding. Reproduced from [52].

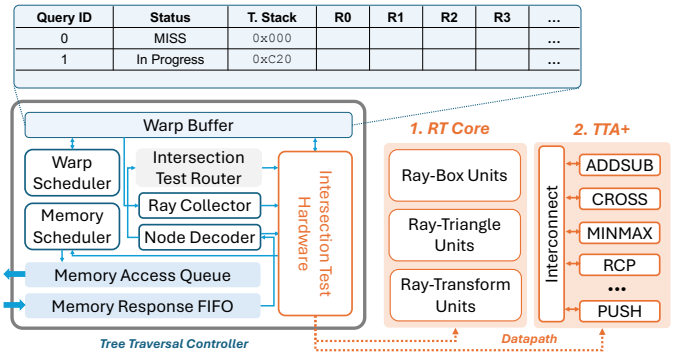


Fig. 5. Ray tracing accelerator architecture on a GPU, which may include fix-function intersection hardware (RT Core design [31]) or programmable operation units (TTA+ design [35]) to execute intersection tests.

early exit opportunities since finding a separating line on any axis guarantees no collision, eliminating the need to check remaining axes. Without early exits, each test requires 30 projections and 15 overlap checks, making collision queries of many robot poses against complex environments in real-time quite challenging.

C. GPU Hardware

GPUs consist of thousands of threads and are designed to accelerate massively parallel applications. Threads in a warp are executed in Single Instruction Multiple Thread (SIMT) fashion, where the same instruction is executed on different data. However, the SIMT architecture does not handle irregular applications well, such as ray tracing, which traverses a BVH tree and performs intersection tests between rays and geometry. Therefore, modern GPUs include a specialized Ray Tracing Accelerator (RTA) to address these performance bottlenecks [31], [53]–[56]. GPUs also include a neural accelerator, like the Tensor Core, that efficiently perform matrix multiplications in neural network inference.

Figure 5 shows the RTA architecture on the GPU. RTAs can be broken down to two main components, the *tree traversal controller* that coordinates with the main GPU cores and fetches required data from memory, and the intersection units forming a *datapath* for computation. When threads in a warp issue a `traceRay` instruction to the RTA, the per-ray information such as ray origin, direction, and a traversal stack are stored in the warp buffer. Every cycle, the warp scheduler selects a warp to execute and issues memory requests to fetch the required tree nodes from memory. Once the data returns, the rays and node data are routed based on the node type to the corresponding intersection units. The intersection units can be implemented as fixed-function hardware, the typical design for ray tracing, or as programmable operation units (OP units) as proposed in the tree traversal accelerator (TTA+) design [35]. TTA+ targets flexibility, with a modular system that decomposes intersection units into individual OP units and connecting them through an interconnect similar to a dataflow architecture [57]. TTA+ executes a custom instruction set called μops, where each μop corresponds to a specific OP unit and an OP destination table specifies which OP unit the output data should route to next. However, TTA+ was not designed for instruction sequences as complex as the SACT and has no ability to support early exits.

Although RTAs are designed for computer graphics, they are often repurposed for other applications such as FastRNN [58] and RTNN [59] for nearest neighbor search, Mochi [32] and RT-DCD [34] for collision detection, and many others [60]–[62]. These works reformulate key algorithms into the ray tracing paradigm to run on RTAs, representing data as 3D points or volumes stored into a BVH structure and issuing "rays" to query the data. For example, RTNN represents data points as spheres, then issues rays to find the nearest neighbors by performing ray-sphere intersection tests. However, these reformulations cause redundant handoffs to non-accelerated intersection shaders that hinder RTA benefits.

III. RoboCore Architecture

Our goal is to design a hardware unit for a RoboGPU that accelerates collision detection with the ability to adapt as robotics workloads inevitably evolve. Ideally, this unit requires minimal hardware modifications to reduce design and validation costs, making it a practical implementation for GPU vendors. Existing RTAs provide an excellent platform for tree traversals, but they are inefficient for collision detection. This section addresses the limitations of RTA designs, including the TTA+, and introduces our RoboCore architecture.

A. High Level Design

Figure 6 illustrates the staged collision detection process that executes the 15-axis SACT, which is the main algorithm that RoboCore targets. The SACT workload exhibits short but highly repetitive instruction sequences that occur in bursts across a large number of independent queries, which is ideal for a fixed-function accelerator. However, robotics pipelines evolve quickly, so a purely fixed-function collision unit would be too rigid to support new emerging kernels and becomes obsolete over time. Meanwhile, a fully programmable CPU-like core is flexible but incurs unnecessary overheads with instruction fetch and decode since SACT repeats the same operation sequence for every query. Dataflow and CGRA-style architectures face other inefficiencies with token tracking and coordinating early exits between independent queries. The closest existing solution that balances programmability and efficiency are RTAs, which take advantage of fixed-function

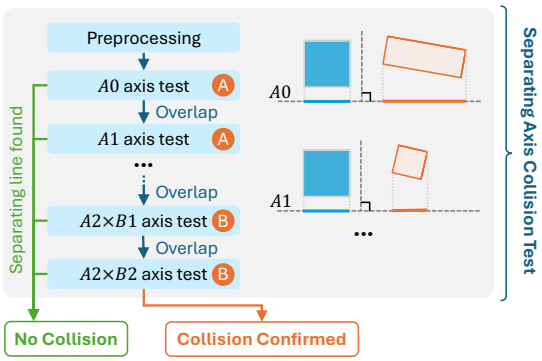


Fig. 6. Staged collision detection with early termination. Box-Normal axis tests executes first, marked with (A), then Edge×Edge axis tests marked with (B). Reproduced from [52].

TABLE I
μOPS USED FOR SACT INTERSECTION PROGRAM

Test type	# of tests	TTA+ μops used per test
Pre-Process	1	4 VEC3-SUB, 1 R-XFORM
A Box-Normal Axis	6	2 DOT, 1 VEC3-SUB, 1 VEC3-CMP
B Edge×Edge Axis	9	1 CROSS, 1 VEC3-CMP

datapaths but are general enough to support computer graphics as the field grows.

Programmable RTAs such as the TTA+ can execute SACT using the list of μops in Table I. However, the complete SACT requires a total of 47 μops to test all 15 axes, which is significantly more than the original TTA+ workloads that required 12 μops on average to 19 μops at maximum per intersection program [35]. Unlike dedicated collision detection accelerators such as MPAccel [52], TTA+ has no mechanism to skip unnecessary axes checks using early exits, which is crucial for performance and would reduce computation latency by 65% on average per query. RoboCore addresses this gap by specializing the intersection datapath for SACT and adding early-exit control while preserving programmable execution for non-SACT kernels. This includes ray tracing kernels if needed since ray tracing does occasionally appear in robotics [33], [63]. Outside the intersection path, RoboCore reuses the same tree traversal controller as RTAs, which integrates well into existing GPU architectures. Rather than inserting an entirely new accelerator, reusing the proven RTA infrastructure where possible lowers the barrier of extending GPU capabilities and amortizes hardware investments through flexibility across applications as they evolve.

B. RoboCore Datapath

RoboCore adds three new four-stage pipelined SACT units (Pre-Process, Box-Normal, and Edge×Edge) that merge TTA+ μop sequences into direct computation per test type, as listed by the SACT decomposition in Table I. Figure 7 shows the RoboCore datapath, which combines the new SACT units with the existing programmable TTA+ model. Beyond the redesigned datapath, we reuse the baseline RTA traversal hardware for RoboCore, including node fetching, traversal-state tracking, and traversal scheduling, which are already heavily optimized. This choice reduces integration overheads and allows RoboCore to benefit from orthogonal traversal optimizations such as treelet prefetching and more [64]–[66].

Each SACT unit features a compute pipeline composed of floating point arithmetic elements such as multipliers, adders, and comparators to execute the required operations for the tests in Table I. Compared to the TTA+ implementation, these SACT units are able to condense and parallelize the operations for each test, which reduces the area overhead of the specialized hardware while also reducing the total latency per test. Each unit still exposes an embedded general μop (R-XFORM, DOT, and CROSS), which accounts for the largest portion of the SACT units and can be reused in other robotics kernels despite the specialized design. The SACT unit also includes the same input buffer, input decoder, output encoder, and output buffer as a standard TTA+ OP unit for compatibility. These components allow the SACT units to share the same interconnect packet format used in TTA+ that can be passed between units and communicate with the main traversal controller.

The SACT units are connected in series to mimic the 15-axis test sequence in Figure 6, with the Pre-Process unit feeding into the Box-Normal unit, which feeds into the Edge×Edge unit. To execute the SACT program, the traversal controller issues a packet to the Pre-Process unit, which executes the first stage of the SACT then autonomously forwards the results to the next SACT unit without going back through the interconnect, which is a source of overhead on TTA+. Alternatively, to use the SACT unit as a general μop, any OP unit can issue a packet directly through the interconnect to any of the three SACT units, which can be configured to only execute the embedded μop bypassing any SACT-specific logic in the datapath. The OP unit then sends the results back through the interconnect to the next unit based on the configuration registers. At the datapath boundary, each unit accepts packets either from the interconnect (programmable mode) or from the previous SACT stage (specialized mode). An input arbiter prioritizes in-flight SACT packets to preserve collision detection latency, while programmable packets may experience some delay since they can tolerate higher latency. This dual-mode behavior lets RoboCore amortize the area of specialized hardware while keeping the same programming model and interconnect protocols for other robotics kernels.

1) Early-Exit Control: To support early exits from the SACT program, we insert control logic (CTRL) in RoboCore between the three SACT units. The controller tracks per-query progress using query IDs and counters for the 15 axes, which are incremented when an axis test passes and reset when a new query is issued. The Box-Normal counters tracks the first six axes tests, while the Edge×Edge counters tracks the remaining nine axes. The controller monitors the SACT units outputs and routes the operation based on the results and the counter state. Figure 8 illustrates an example of the control logic for the Box-Normal unit, showing routing for four different queries

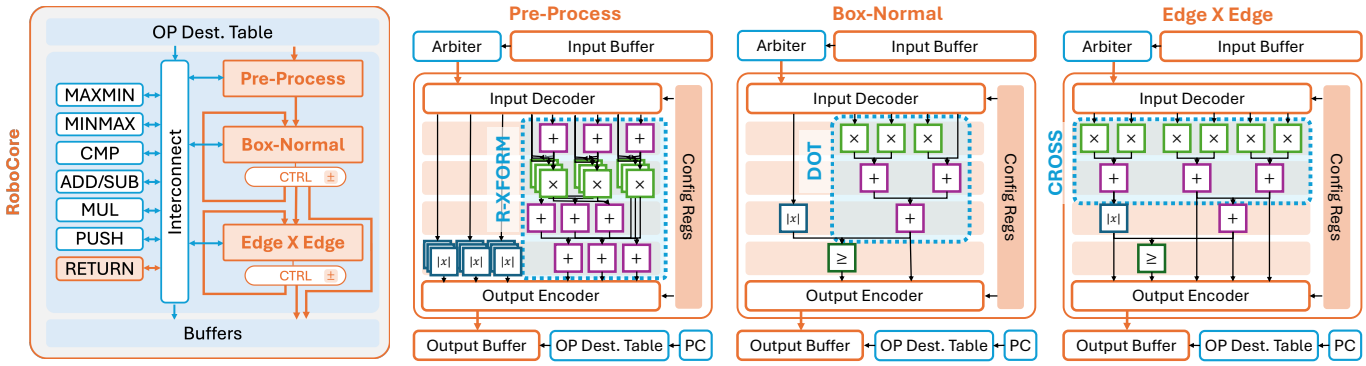


Fig. 7. RoboCore datapath using three specialized SACT OP units with embedded general μops.

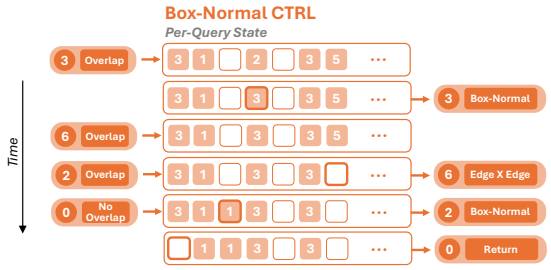


Fig. 8. Example of CTRL logic for Box-Normal unit.

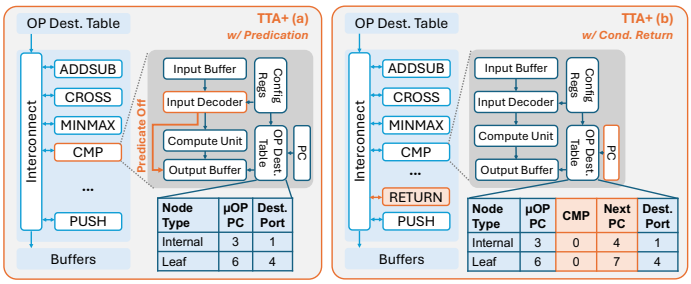


Fig. 9. Early exit support on TTA+ with (a) predication and (b) conditional returns. Modifications are in orange.

over time. If the output is a separating axis, the controller terminates the program (i.e. Query 0), sending the results back to the traversal controller and updating the warp buffer status. Otherwise, the controller returns the operation to the same SACT unit for the next axis test (i.e. Query 2 and 3), or forwards the operation to the next SACT unit when the counter overflows (i.e. Query 6). This control mechanism allows for early termination of non-colliding queries and reduces interconnect traffic. We also include a new RETURN OP unit in the programmable datapath that can be used to signal early exits for non-SACT kernels, which is described in Section III-C2.

C. Early Exit Support for Non-SACT Kernels

The RoboCore datapath adds early exits for the SACT kernel, but early exits are beneficial in other robotics kernels as well. Figure 9 illustrates two possible approaches to support non-SACT μop programs generically applicable to TTA+ architecture: predication and conditional returns.

1) Predication: Predication requires minimal hardware modifications by tagging each μop with a predicate flag stored as part of the interconnect packet. The input decoder uses this bit to control whether the operation is executed or bypassed directly to the output buffer. The CMP (compare) unit updates the predicate based on comparison outcomes, similar to control tokens in dataflow machines [57]. Although predication can suppress arithmetic work, all μops still need to be decoded and travel through the interconnect, which still incurs an overhead and wastes energy. Section V-A evaluates the performance and inefficiencies of predication.

2) Conditional Returns: A better approach is using conditional returns, where CMP redirects control flow like a switch operator in dataflow machines [67] and skips the remaining program. To support this behavior, we add a RETURN OP unit (Figure 9 (b)) that terminates the intersection program and signals back to the traversal controller that traversal can continue. The same RETURN OP unit is included in RoboCore. Each OP unit in TTA+ relies on an OP destination table to specify where the output data should be routed next, but the RETURN OP unit does not need one since it always signals the end of the program. In the CMP OP unit, we extend the OP destination table with a column for the compare result to create two entries per μop PC for true and false outcomes that route to different next OP units. The PC counter inside CMP is modified to read the next PC value from the OP destination table instead of simply incrementing. These changes support conditional branching in general beyond just returns, which opens up possibilities for more complex intersection programs in the future. Section V-A compares the performance of conditional returns versus predication.

D. Generalizing RoboCore

Other robotics algorithms with similar GPU inefficiencies can also be mapped to the RoboCore functionality. RoboCore supports any kernel that can execute on RTAs or TTA+, which already includes a wide range of tree traversal and pointer-

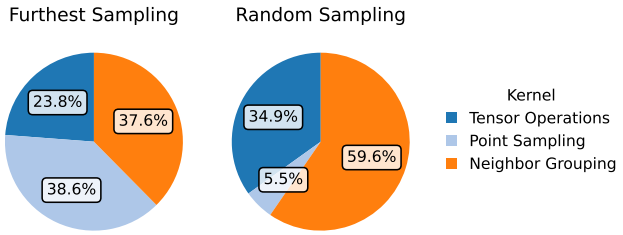


Fig. 10. Latency distribution of MπNet [2] using furthest point sampling and modified random point sampling.

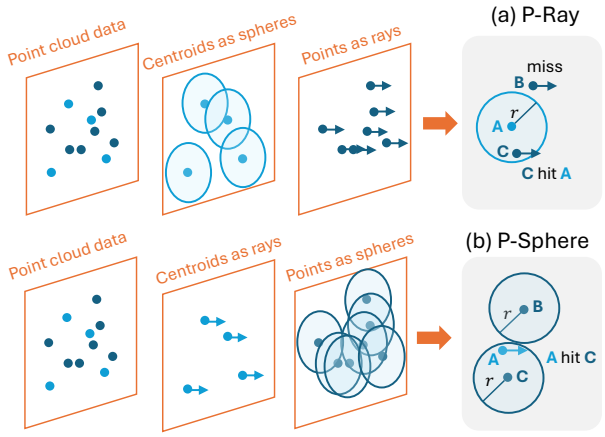


Fig. 11. Approaches for applying ray tracing to PointNet++. (a) P-Ray: sampled points as spheres and others as rays. (b) P-Sphere: sampled points as rays and others as spheres.

chasing algorithms. One important example non-SACT kernel that RoboCore accelerates is used in point cloud processing, which motion planners rely on to understand the environment. Point clouds are obtained from sensors such as LiDAR then processed through networks like PointNet++ [45] to extract features. Figure 10 shows the latency distribution of a single MπNet [2] forward pass executed on a GPU, which uses PointNet++ as the backbone. The neighbor grouping kernel is the biggest bottleneck overall, consuming 59.6% of the total inference time for random sampling and 37.6% for furthest point sampling. This kernel is executed as a ball query operation that identifies neighboring points within a certain radius (ball) of the query, which can already be reformulated to RTAs as described in RTNN [59] and FastRNN [58] using spheres and rays. Although the reformulated RTA implementation performs better than CUDA, custom intersection programs for ray-sphere tests and the lack of early-exit support incur high overheads on RTAs. Fortunately, both issues are resolved by RoboCore, meeting our goal of an accelerator that adapts to different robotics kernels.

There are two different ways to execute ball query on RoboCore since the operation is commutative [58]. Figure 11 illustrates the two approaches (P-Ray and P-Sphere), where P-Ray models the sampled points as spheres and other points as rays, and P-Sphere models the sampled points as rays and other points as spheres. For example, if point A is the sampled point, and the goal is to find if point B or point C is within the radius of point A, then P-Ray creates a sphere around point A and traces infinitely short rays from point B and point C to check for intersections, while P-Sphere creates spheres around points B and C and traces infinitely short rays from point A to check for intersections. In both cases, the result is that ray A hits sphere C or ray C hits sphere A, implying that point C is within the desired radius of point A. Although FastRNN introduces both configurations, the authors only evaluate P-Ray based on the assumption that RTAs favor P-Ray [58]. With the added support in RoboCore, we expect P-Sphere surpasses P-Ray because more spheres are built into highly efficient bounding volume hierarchies (BVH) structures without additional ray-sphere test overheads and can be skipped using early exits once enough points are identified. We evaluate both configurations in Section V-B1.

E. RoboCore Programming Model

Programming RoboCore is similar to programming existing RTAs, in which intersection programs are pre-configured to the device then invoked during tree traversal. The RoboCore datapath can be configured to execute either the default SACT program or custom μop sequences that allow developers to implement new kernels as needed. Table II summarizes the API for programming RoboCore and Table III lists the available μops with their associated OP unit and description. RoboCore functions are invoked within a GPU kernel where each thread represents a single collision query. The programmer needs to configure RoboCore to decode the data layout of the OBBs and octree nodes using the `ConfigDecode` API. Then, each thread executes `CheckCollisions`, which launches the SACT program on RoboCore and returns the collision results after completion. Listing 1 summarizes the API usage with pseudocode for collision detection on RoboCore, which directly integrates with CUDA kernels for general-purpose programming, unlike TTA+ that uses the Vulkan graphics pipeline. For other programs, the host first configures the desired datapath on RoboCore by sending a μop assembly file that specifies the sequence of operations and their configurations, much like the TTA+ programming model.

Another benefit of the RoboCore programming model is the ability to track tree traversal metrics such as the number of nodes visited. This feature is enabled by decoupling RoboCore from the graphics paradigm and allowing additional variables to be pushed to the warp buffer through the PUSH μop. We demonstrate how tracking these metrics can be used to further optimize traversal performance by adjusting traversal strategies dynamically based on program characteristics at runtime in Section V-C.

IV. Methodology

We evaluate our proposed RoboCore architecture using Vulkan-Sim [68], a cycle-level GPU architecture simulator commonly used to evaluate ray tracing hardware [35], [65],

TABLE II
ROBOCORE PROGRAMMING API

Type	API / Description
Host	`ConfigRoboCore("collision_alg.asm")` Configure desired datapath on RoboCore using μop assembly file.
Host	`ConfigDecode(Query, NodeA, NodeB)` Configure decode units on RoboCore for query, node type A (e.g. inner nodes), and node type B (e.g. leaf nodes) layouts.
Device	`CheckCollisions(OBB, TreeRoot, Results)` Defaults to SACT algorithm. Executes collision detection on the input OBB and octree.
Device	`ExecuteRoboCore(Query, TreeRoot, Results)` Runs configured datapath (e.g. ball query) on the input query and tree structure.

TABLE III
SUPPORTED ROBOCORE μOPS

OP Unit	Supported μop	Description	Latency
ADDSUB	ADD, SUB	FP32 Vec3 addition and subtraction.	3 cycles
MINMAX	MINMAX	FP32 MIN(a, MAX(b, c)).	2 cycles
MAXMIN	MAXMIN	FP32 MAX(a, MIN(b, c)).	2 cycles
CMP	CMP	Per-element Vec3 comparison.	1 cycle
MUL	MUL	FP32 scalar multiplication.	5 cycles
Pre-Process	PRE, XFORM	SACT preprocessing operations, ray transform matrix multiplication.	14 cycles
Box-Normal	BOX, DOT	SACT Box-Normal operation, FP32 Vec3 dot product.	12 cycles
Edge×Edge	EDGE, CROSS	SACT Edge×Edge operation, FP32 Vec3 cross product.	12 cycles
LOGIC	AND, OR, XOR, NOT	Logical operations.	1 cycle
PUSH	PUSH	Push to warp buffer or traversal stack.	1 cycle
RETURN	RETURN	Exit from OP units.	1 cycle

[69]–[73]. We rely on Vulkan-Sim to measure RoboCore performance since RoboCore is not in any existing robots and robotics simulators such as Rosé [74] are not designed to evaluate GPU systems. Table IV shows the hardware configurations used in our evaluation, matching TTA+. We model RoboCore with SACT units in Vulkan-Sim to include our architectural changes. We evaluate energy consumption of RoboCore using AccelWattch [75] integrated with Vulkan-Sim and the OP units in RoboCore by tracking active cycles and operations performed by each unit. To estimate the area overhead, we implement all OP units using Chisel [76], with Berkeley HardFloat [77] for floating point operations, and synthesize with Synopsys Design Compiler using the FreePDK 45nm standard cell library [78]. The synthesized design is used to estimate latency for each SACT unit in the simulator.

A. Evaluation Workloads

We evaluate the performance of RoboCore across several robotics workloads, each using their associated environment datasets, and compare against baselines described in this section.

1) Environments: We evaluate collision detection on four different environments with varying complexity and density of obstacles: Cubby, Dresser, Merged Cubby, and Tabletop.

```
// Define OBBs and octree structures for robot poses and the environment
OctreeNode* treeRoot = BuildOctree(EnvironmentPointCloud);
OBB* obbs = CreateOBBs(RobotPoses);
// Copy to device and allocate results buffer
CollisionResult* d_results = AllocateDeviceBuffer();
OBB* d_obbs = CopyToDevice(obbs);
OctreeNode* d_treeRoot = CopyToDevice(treeRoot);

// Configure OBB and octree node layouts (byte offsets)
size_t obbLayout[5] = {0, 6, 12, 18, 24};
size_t nodeLayout[3] = {0, 2, 10};
ConfigDecode(obbLayout, nodeLayout, nodeLayout);

// Define kernel for collision detection
__global__ void CollisionKernel(OBB* d_obbs, OctreeNode* d_treeRoot,
      CollisionResult* d_results) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    CheckCollisions(d_obbs[idx], d_treeRoot, &d_results[idx]);
}

// Launch kernel with appropriate dimensions and synchronize results
CollisionKernel<<<blocks, threadsPerBlock>>>(d_obbs, d_treeRoot, d_results);
cudaDeviceSynchronize();
CollisionResult* h_results = CopyToHost(d_results);
```

Listing 1. Programming RoboCore for collision detection.

TABLE IV
VULKAN-SIM CONFIGURATIONS

# Streaming Multiprocessors (SM)	8
Max Warps / SM	32
Warp Scheduler	GTO
# Registers / SM	32768
Instruction Cache	128KB, 16-way assoc., 20 cycles
L1 Data Cache + Shared Memory	64KB, Fully assoc. LRU, 20 cycles
L2 Unified Cache	3MB, 16-way assoc. LRU, 160 cycles
Compute : Interconnect : L2 : Memory Clock	1365 : 1365 : 1365 : 3500 MHz
# RoboCore Units / SM	1
Warp Buffer Size	4 warps
# RoboCore Intersection Units	4 sets

These environments are taken from MπNet [2] and each environment is paired with OBBs enclosing different robot poses along planned trajectories. The Tabletop environment is the simplest structure, but still features randomly placed obstacles that create a cluttered environment. The Cubby, Dresser, and Merged Cubby environments have a more complex structure, with smaller openings that require precise motion planning. To compare against MPAccel [52], we additionally evaluate ten environment scenarios with 100 pairs of start and end goals per environment based on data published by the authors. We also evaluate performance with Alpamayo-R1 [3] as a representative VLA model, using the public 10B parameter model with randomly selected clips from Physical AI AV Dataset [79], which provides LiDAR data that we use for collision detection. These environments are static during the collision query and we leave dynamic obstacles to future work.

2) Collision Detection: MπNet does not include collision detection in their code even though the authors acknowledge that it should be combined for safe operation in real robot systems, so we implement our own OBB-octree collision queries. We use the same programming model as TTA+, with each thread representing a single OBB-octree collision query. We compare the performance of RoboCore against a CUDA-based GPU implementation [52] that also checks for collisions by testing OBBs against an octree data structure as our baseline. We also compare against Mochi [32], which maps collision detection to RTA hardware on GPUs. However, Mochi is designed for particle-based simulations that track all collisions, whereas motion planning applications only need to detect collisions between the robot and the environment. Therefore,

we modify Mochi to filter out self-collisions using the RTA programmable intersection shaders. We build a BVH tree using point cloud data of the environment and robot OBBs with Embree [80], implement Mochi using the Vulkan API [81], and evaluate its performance using Vulkan-Sim. Lastly, we compare against CuRobo [82], an optimized CUDA robot library that includes collision detection. However, CuRobo employs spheres instead of OBBs for its robot representation even though it is less accurate and results in more false positive collisions. We implement a sphere-based collision detection on RoboCore using our programmable μops to compare against CuRobo as a reference.

3) Overall Neural Planning: We evaluate the performance of MπNet [2] with acceleration from RoboCore on the ball query kernel in PointNet++ [45], comparing to the RTNN implementation provided by the authors [59]. We use the same PointNet++ configurations as implemented in MπNet to ensure a fair comparison. For measuring the latency of other kernels that do not require hardware modifications, we use the CUDA implementation provided by the authors of MπNet [2], measured on a NVIDIA RTX 5070 Ti GPU. We also evaluate RoboCore with MPNet [1] despite its lower success rates to create a direct performance comparison against other hardware platforms, including MPAccel [52] (standalone accelerator) and P3NetCore [83] (FPGA). To our knowledge, there are no other hardware solutions that target neural planners with explicit collision detection.

4) Classical Motion Planning: Although RoboCore is designed for collision detection and neural motion planning, we also evaluate its performance on a classical algorithm from the RoWild benchmark [36]. RoWild includes a set of environments and tasks commonly encountered in robotics applications, such as navigation, manipulation, and perception. While most kernels in RoWild are already efficiently parallelized for GPUs, we find that some irregular kernels can benefit from RoboCore. For example, DeliBot uses a ray casting algorithm for Monte Carlo Localization that accounts for 74% of end-to-end latency. We modify the localization kernel to use the RoboCore architecture for ray casting, which traverses by stepping along the ray rather than down a tree and checks for collisions by accessing the occupancy grid of the environment.

V. Results

We find that a RoboGPU with RoboCore effectively accelerates collision detection and overall neural planning compared to a baseline GPU system with an RTA. In this section, we present our evaluation results for RoboCore.

A. Collision Detection Performance

RoboCore achieves 13.3× and 2.8× speedup on collision detection queries over the CUDA and Mochi baselines, respectively, as shown in Figure 12. Although Mochi already accelerates collision detection using RTAs, its performance suffers from many unnecessary collision checks and the cost of invoking the intersection shader at each hit. In contrast, the

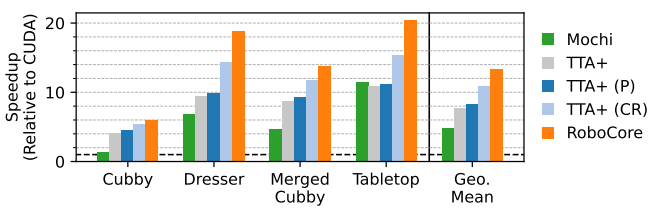


Fig. 12. Speedup of collision detection on RoboCore relative to CUDA baseline, with comparisons to Mochi on RTAs, baseline TTA+, and TTA+ modifications for predication (P) and conditional returns (CR).

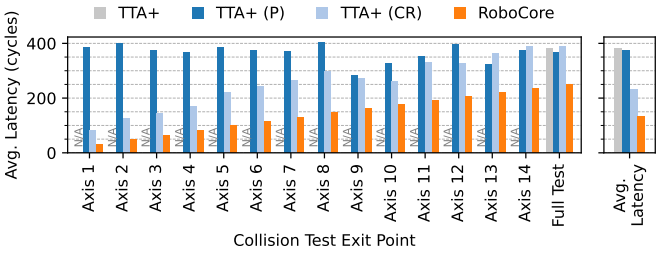


Fig. 13. Latency distribution of collision detection queries for different exit conditions. TTA+ only performs Full Test.

RoboCore architecture is optimized for collision detection and avoids overheads that are created by forcing queries through the ray tracing pipeline. Early termination and collision units within RoboCore also leads to significant improvements over the TTA+ architecture. Figure 13 shows the latency distribution of collision queries for different exit conditions, highlighting the benefits of early termination. Predication is an ineffective early termination option since μops must still travel through the interconnect and OP units, making almost no impact on the average collision latency. Conditional returns (CR) eliminate this overhead, providing 40% speedup over TTA+ alone. Finally, dedicated SACT units on RoboCore create an additional 22% speedup over TTA+ (CR), benefitting from specialization while preserving flexibility.

Furthermore, Figure 14 compares the performance impact of different SACT unit latency, adjusting by 0.5× to 2× on all SACT units. The normalized execution time of collision detection shows only small performance variations, implying insensitivity to individual SACT unit latency and suggesting that the SACT units can be designed to conserve area and energy instead of optimizing for latency despite collision detection being a time-sensitive task. Figure 14 also compares RoboCore to a parallel SACT design, where six Box-Normal and nine Edge×Edge units are used to perform all 15 axis tests simultaneously. While latency is reduced by 41%, the area is nearly 3.7× larger. If latency is prioritized over hardware area, a better design point could include two of each SACT unit in RoboCore, which results in 1.2× speedup at 1.8× the area of the default option and maintains early exit energy savings.

RoboCore can also accelerate other scenarios, such as smaller MPAccel environments [52] and Physical AI AV

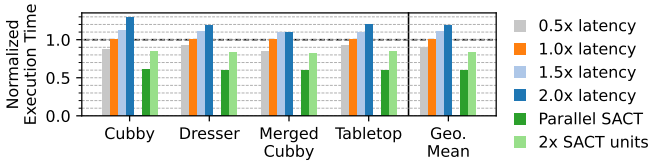


Fig. 14. Relative impact of SACT unit configuration on overall collision detection performance.

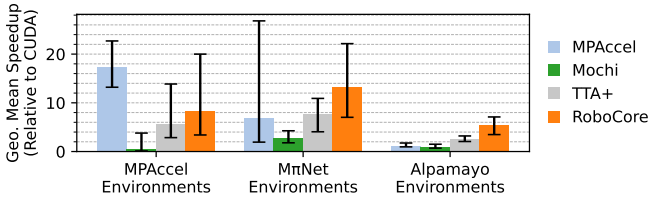


Fig. 15. Performance results of collision detection for MPAccel test scenarios compared to MπNet and Alpamayo-R1 environments, showing average, min, and max speedup.

scenes [79] with simpler robot geometries, supporting a range of real-world behavior. Figure 15 compares RoboCore to MPAccel. Although RoboCore cannot match MPAccel on their environments, which reports 23× speedup using 16 collision units over the same CUDA baseline for 2^{20} collision queries, RoboCore benefits from a more sophisticated memory system and eliminates IO overheads that standalone accelerators incur. Figure 15 also evaluates RoboCore on randomly selected scenes from Physical AI AV [79] used by Alpamayo-R1 [3]. Vehicles have fewer degrees of freedom than the robotic arms in MπNet but RoboCore is still 5.4× faster on average, generalizing better to different use cases than MPAccel.

Using sphere-based robot representation, RoboCore achieves 21× speedup over the OBB-based CUDA baseline and is faster than CuRobo [82]. CuRobo latency ranges from 4.6-9.9μs per robot pose, compared to 0.18-0.36μs on RoboCore for the same MπNet environments. However, CuRobo does not use an octree structure and computes a complex distance metric in collision detection, accounting for its higher latency. Regardless, we find that sphere approximations results in 10% false collisions, causing some feasible trajectories to be rejected. Unnecessary replanning is then required, leading to worse overall performance for the motion planner.

The performance improvements of RoboCore directly translates to energy savings, which is a key factor in robotics. Figure 16 shows the energy consumption results of RoboCore for collision detection queries compared to the CUDA and Mochi baselines. Energy from the main GPU components dominate the total energy consumption, with the RoboCore only contributing to less than 5% of the total energy on average. We achieve 87% energy savings against the CUDA baseline and 48% against Mochi by reducing redundant operations and improving overall performance.

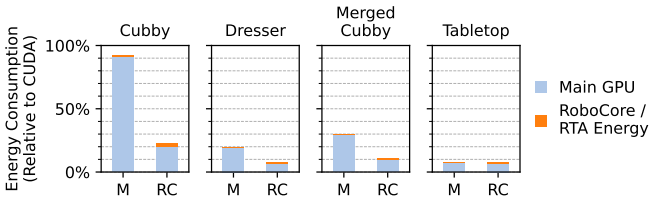


Fig. 16. Energy consumption of collision tests for Mochi (M) and RoboCore (RC).

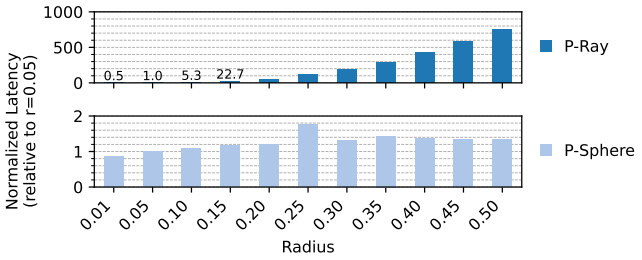


Fig. 17. Execution latency of different grouping radii (r) on ball query with RoboCore, relative to r=0.05 used by MπNet.

B. Neural Planning Performance

RoboCore outperforms existing point cloud processing implementations such as RTNN, which compounds with the fast collision detection to accelerate end-to-end robotics motion planning.

1) Point Cloud Processing: We find that the CUDA implementation of ball query used in PointNet++ is inefficient and RTNN provides a more reasonable baseline. RTNN has been shown to reach 44× speedup on average even over the highly optimized cuNSearch [84] implementation. RoboCore further outperforms RTNN by 1.9× on average using P-Sphere, which is only made possible by our proposed architectural optimizations. P-Sphere is enhanced by the programmability and early termination support in RoboCore, which avoids the overhead of running custom intersection shaders in the compute cores and unnecessary intersection tests beyond the required number of spheres. Table V compares P-Ray and P-Sphere for the Cubby environment to quantify this difference. P-Ray generates a large number of rays that traverse through a small tree of spheres with six levels, whereas P-Sphere generates a small number of rays that traverse through a much larger tree of spheres with twelve levels. Surprisingly, P-Sphere performs better despite only launching enough rays to utilize 47.4% of the RoboCore occupancy. This result is because the tree structure organizes the spheres for a very efficient traversal, leading to fewer memory fetches for node data overall. In the RTNN implementation, there is no mechanism to immediately stop traversal after a maximum group size is reached, forcing the hardware to continue traversing the tree and launching shader invocations simply to find that no more points are needed. This problem is much worse with P-Sphere because

TABLE V
COMPARISON OF P-RAY AND P-SPHERE ON POINTNET++ FOR CUBBY ENVIRONMENT WITH ROBOCORE RELATIVE TO RTNN.

	P-Ray	P-Sphere
Total rays	523,776	512
RoboCore occupancy	100%	47.4%
Total spheres	512	523,776
Tree depth	6	12
Avg. (nodes, spheres) / ray	(14.1, 2.3)	(959.2, 276.1)
Total nodes traversed	6,856,980	349,272
Speedup	1.19×	2.93×
Avg. speedup (all env.)	1.17×	1.95×

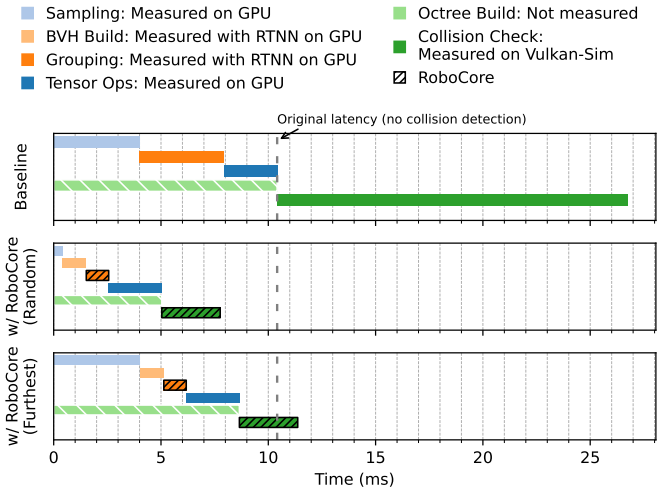


Fig. 18. Latency breakdown of proposed motion planning pipeline on GPU with RoboCore compared to the baseline CUDA implementation for Cubby environment.

the number of spheres is much larger than the maximum group size. On average, RoboCore traverses 6× fewer nodes with early exit than without, which leads to a significant reduction in latency and energy consumption.

To ensure these results generalize beyond MπNet, we also sweep different radius values that may be used by PointNet++ in other robotics workloads like ContactGen [85], Sugar [86], HDP [46] or others. Figure 17 shows that P-Ray does not scale well with larger radius values and the relative latency grows exponentially alongside the increased intersection shader invocations. P-Sphere, which is enabled by RoboCore, scales much better.

2) Full Motion Planning Pipeline: By accelerating point cloud processing in PointNet++, we balance neural inference and RoboCore tasks, allowing better utilization of the specialized GPU resources in an end-to-end motion planning pipeline. Figure 18 shows the latency breakdown of MπNet measured on a GPU, including the neural motion planner and explicit collision detection. Since RoboCore cannot be measured on existing GPUs, we estimate the BVH Build and Grouping times using RTNN and the Collision Check time by applying our measured speedup over CUDA. Using random sampling in PointNet++, combined with neighbor grouping on

TABLE VI
OVERALL PLANNING LATENCY FOR MPNET (INCLUDING COLLISION DETECTION) ON DIFFERENT DEVICES.

Hardware	Latency (ms)
NVIDIA RTX 3090	21
NVIDIA Jetson TX2	110
P3NetCore (FPGA)	50
MPAccel (standalone accelerator)	1.1
GPU w/ RoboCore	7.5

RoboCore, we accelerate the point cloud processing stage in MπNet by 2.2× over the CUDA baseline. With this speedup, the additional latency from explicit collision detection does not delay the overall motion planning pipeline and actually completes 1.3× faster on our RoboGPU with RoboCore than the baseline while also reducing the 14% collision rate to 0%. Although PointNet++ uses furthest point sampling by default, we find that random point sampling can achieve an 88.7% success rate, which is only slightly lower than the 94.8% of furthest point sampling, but saves 29% of the total latency. This result is important as it has been shown that lower latency can be more important than higher accuracy for overall robotics performance [74]. The full pipeline is 3.4× faster and meets real-time latency requirements for robotics, overcoming the limitations described in MπNet for enabling safer operations.

Octree construction can begin as soon as the point cloud is available, in parallel with other motion planning tasks. The total latency is dependent on the speed of the octree builder, which only becomes the critical path if it is slower than the sensor frequency when combined with collision checking. However, there are many orthogonal works optimizing octree construction and updates [33], [48]–[50] that can be integrated, which is beyond the scope of RoboCore. After the first iteration octree build, subsequent octree updates are substantially faster and can update at around 20Hz [49] compared to the 10Hz LiDAR sensor [79], [87]. Collision detection and milestone generation in MπNet may also be pipelined together using accelerator-level parallelism between the RoboCore and a tensor accelerator to further reduce latency.

To compare end-to-end performance against other hardware designs, Table VI presents the total planning latency of MPNet on different devices [52], [83] against a GPU with RoboCore. RoboCore is faster than both baseline GPUs and an FPGA accelerator (P3NetCore), highlighting the benefits of specialization. Although MPAccel is faster, it potentially underestimates coordination overheads between the DNN accelerator, CPU controller, and MPAccel that RoboCore does not have. MPAccel also relies on strategic scheduling that is only beneficial when a collision exists, which becomes less likely as neural planner improve. Both P3NetCore and MPAccel lack programmability compared to GPUs and RoboCore, executing only MPNet and P3Net, which are very small MLP models with poor success rates compared to newer neural planners.

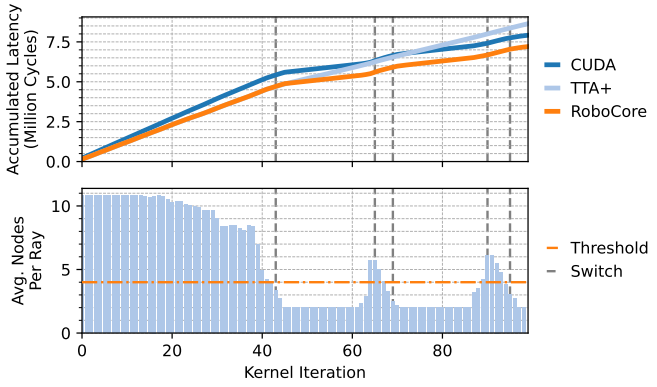


Fig. 19. Execution time of DeliBot from RoWild on RoboCore compared to CUDA and TTA+ implementations.

TABLE VII
AREA BREAKDOWN OF COMPUTATION IN ROBOCORE UNITS (R) COMPARED TO TTA+ (T) AND MPACCEL.

	Unit	Area (μm^2)	Area Overhead
R	Pre-Process	223911	41054 (+22%)
T	(R-XFORM)	182857	
R	Box-Normal	78381	15112 (+24%)
T	(DOT)	63269	
R	Edge × Edge	121284	18473 (+18%)
T	(CROSS)	102811	
R,T	Vec3 ADD	36159	
R,T	MUL	15959	
R,T	Vec3 CMP	11595	
R,T	MINMAX	7120	
R,T	MAXMIN	7190	
	TTA+ Total	**426960**	
	RoboCore Total	**501599**	**74639 (+17%)**
	MPAccel	**251000**	**251000 (+59%)**

C. Classical Algorithm

We demonstrate that RoboCore can improve classical motion planning when neural planners are not suitable using an example from RoWild [36]. Figure 19 shows the cumulative execution time of the DeliBot kernel on RoboCore compared to CUDA and TTA+ implementations, with 10% and 20% speedups respectively. RoWild benchmarks execute over a large number of kernel iterations, exhibiting different characteristics throughout execution. For example, DeliBot begins with randomly initialized particles that are refined over time. To start, there are few collisions, leading to long ray traversals ideal for RoboCore. However, as the particles converge, the number of collisions increase, leading to short traversals that fail to offset the RoboCore launch overhead. Therefore, we allow DeliBot to dynamically switch between execution on the RoboCore and CUDA cores for each kernel iteration, based on expected collisions. We implement this dynamic switching by using a threshold on average traversal distance in the previous iteration, tracked using the average number of nodes traversed per ray. Figure 19 also shows how this metric changes over time, triggering the switches between the RoboCore and CUDA cores, resulting in the best overall performance. This result reinforces the importance of placing RoboCore within the GPU to reap the benefits of both programmability and specialization without incurring orchestration overheads.

Other RoWild kernels like CarriBot and MoveBot can also benefit from RoboCore. CarriBot uses an A* algorithm that relies on collision detection and MoveBot uses a RRT* algorithm that implements a nearest neighbor search, both of which RoboCore can accelerate. RoboCore can also still be used to accelerate regular ray tracing workloads, with only a 12% slowdown compared to RTAs when evaluated on LumiBench [88]. This is negligible compared to the 10× speedup achieved by RTAs over software approaches [31].

D. Area Overheads

The area overhead of the modifications in the RoboCore architecture is only 17% more than TTA+ since the SACT units are designed to be dual-purpose. Conditional returns only require an increase to the size of the operation destination table, where each entry is just 2 bytes. Although we add an additional operation unit for RETURN, the additional port does not increase the area overhead because TTA+ already supports up to 16 ports to the interconnect. Alternatively, the design from MPAccel [52] for collisions only requires an area of 0.251mm^2, which is an additional 59% overhead on top of the TTA+ in order to provide the same generality as RoboCore. Furthermore, MPAccel uses fixed-point hardware unlike RoboCore, which may not be effective for collision detection across a wide variety of environment scales and robot sizes. Overall, RoboCore is only 2% larger than an RTA [89] synthesized at the same 45nm process, adding less than 1% area to a robotics GPU such as the NVIDIA Jetson Orin Nano when scaled. This result fits our goal of minimizing modifications to reduce design and validation costs.

VI. RELATED WORK

Computer architecture has an important role in robotics. This section discusses related works optimizing robotics tasks and tree building techniques for environment representation.

A. Accelerating Robotics Workloads

Robotics is a broad field with many different tasks spanning scene representation and motion planning. While there have been many proposed hardware accelerators for specific robotic workloads [52], [90]–[95], these accelerators lack the flexibility to adapt to the rapidly changing landscape of robotics. Tartan [96] proposes a general-purpose CPU architecture optimized for robotics, but evaluated solely on algorithmic motion planning workloads. As the robotics field shifts from traditional algorithmic approaches to neural-based approaches like RT-1 [97] and GR00T [98], GPUs will be a better general-purpose device to support both approaches.

Motion planning and collision detection in robotics are well-studied problems and there have been many works proposing optimizations for these tasks. Amato and Dale [99] first show that motion planning can be parallelized on a multicore CPU, then Gayle et al. [100] show that both path planning and

collision detection can be parallelized on GPUs. Bialkowski et al. [101] used a brute force CUDA algorithm to parallelize collision detection checks on the GPU, achieving massive speedups over the serial CPU implementation. Kider et al. [102] showed how a random search algorithm can be accelerated on a GPU. Pan and Manocha [103] proposed a BVH tree-based GPU collision detection using hierarchical traversal, which is faster than prior GPU implementations. More recently, ReCA [23], Orianna [104], MOPED [24], and Blitzcrank [105] all target a specific motion planning algorithm, but are less flexible than a GPU, especially for neural motion planning. FPGAs [16], [83], [106]–[109] and ASICs [20]–[22], [110], [111] have also been used to accelerate motion planning. However, FPGAs become prohibitively expensive match GPU levels of neural motion planning support with large power gaps to ASICs [27], while ASICs fail to adapt to the wide variety and changing landscape of motion planning algorithms. These solutions are less accessible to the consumer market compared to GPUs, but can be more useful in specialized industrial applications.

Typically in robotics, many different algorithms are running concurrently on the same processor. Although scheduling is outside the scope of this paper, we note that an efficient scheduling algorithm tailored for the modern robotic workloads is crucial for achieving high performance, such as RED [112]. Meanwhile, works such as CoopRT [65] and treelets [64], [69] introduce scheduling optimizations to improve the performance of ray tracing workloads on RTAs, which are likely to be beneficial for RoboCore with robotics workloads as well. Other solutions also optimize neural inferences to balance workloads between accelerators on the GPU in real-time rendering and can be applied to robotics workloads [113].

B. Tree Building Techniques

While tree building is not in the scope of this paper, it is still a crucial part of the collision detection pipeline. Elseberg et al. [114] proposed data structures and algorithms to efficiently construct octrees from point cloud data, such as from laser scanners on robots. They also proposed an efficient octree to store and compress 3D data without precision loss, especially on large point cloud datasets [115]. Su et al. [116] explored an algorithm to perform segmentation on LiDAR point cloud data using octree decomposition. Hao et al. [117] proposed a method to construct octrees from LiDAR data that specializes in detecting changes to buildings and trees in an urban environment. More recently, OctoMap-RT [33] and OctoCache [49] accelerate the OctoMap [118] octree representation build and update operations, while others like i-Octree [119] target SLAM. These works ensure that the octree representation matches the current state of the environment and can easily be used in collision detection. BVH trees are also commonly used to represent the environment, with various construction algorithms such as LBVH [120], PLOC [121], H-PLOC [122], PLOC++ [123], and CWBVH [124] that may offer better performance for some robotics tasks.

VII. Conclusion

In this paper, we present RoboCore, a robotics-focused hardware unit integrated into a RoboGPU architecture to accelerate collision detection and other robotics tasks. Collision detection is crucial in neural planners but requires performance that can be difficult to achieve on GPUs alone. We propose simple yet effective architectural modifications that convert RTAs into RoboCore to be useful in robotics. RoboCore enhances the GPU and achieves 2.8× speedup over the state-of-the-art Mochi [32] implementation for collision detection, while also accelerating MπNet [2] by 3.4× and Monte Carlo Localization from the RoWild [36] by 1.1×. RoboCore explores an architecture that enables the first step towards safe deployment of neural planners, but use on real robotic systems will likely require further improvements before any safety guarantees can be made. Our results show that GPUs with specialized cores offers a flexible and efficient platform that can adapt to the rapidly evolving landscape of robotics while meeting performance demands.

Acknowledgment

The authors would like to thank the anonymous reviewers for their helpful comments. This work was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC). Tor M. Aamodt currently serves as a consultant for NVIDIA. Special mention to Alexander Zheng.

References

[1] A. H. Qureshi, A. Simeonov, M. J. Bency, and M. C. Yip, "Motion planning networks," in *Proc. IEEE Int'l Conf. on Robotics and Automation (ICRA)*, 2019, pp. 2118–2124.

[2] A. Fishman, A. Murali, C. Eppner, B. Peele, B. Boots, and D. Fox, "Motion policy networks," in *Proc. Conf. on Robot Learning (CoRL)*, 2022.

[3] Y. Wang, W. Luo, J. Bai, Y. Cao, T. Che, K. Chen, Y. Chen, J. Diamond, Y. Ding, W. Ding *et al.*, "Alpamayo-R1: Bridging reasoning and action prediction for generalizable autonomous driving in the long tail," *arXiv preprint arXiv:2511.00088*, 2025.

[4] A. Elluswamy, "Building an autonomous future," Keynote, ICCV 2025 Workshop on Distillation of Foundation Models for Autonomous Driving, Talk video: https://x.com/aelluswamy/status/1981760576591393203?s=20, 2025, accessed: 2026-04-01.

[5] L. Miskovetz, B. Bolsens, E. Huang, S. Herrmann, and J. Han, "Actuator and actuator design methodology," Patent WO2 024 072 984A1, 2024. [Online]. Available: https://patents.google.com/patent/WO2024072984A1/en

[6] A. Ogale, M. Bansal, and A. Krizhevsky, "Neural networks for vehicle trajectory planning," Jan. 5 2021, uS Patent 10,883,844.

[7] J. Soares. (2026, January) Mercedes-Benz Unveils New S-Class Built on NVIDIA DRIVE AV, Which Enables an L4-Ready Architecture. NVIDIA Blog, accessed April 2, 2026. [Online]. Available: https://blogs.nvidia.com/blog/mercedes-benz-l4-s-class-drive-av-platform/

[8] R. A. Brooks. (2024) Rodney Brooks' three laws of robotics. Accessed: 2026-04-02. [Online]. Available: https://rodneybrooks.com/rodney-brooks-three-laws-of-robotics/

[9] A. Fishman, A. Walsman, M. Bhardwaj, W. Yuan, B. Sundaralingam, B. Boots, and D. Fox, "Avoid everything: Model-free collision avoidance with expert-guided fine-tuning," in *Proc. Conf. on Robot Learning (CoRL)*, 2024.

[10] T. M. Austin, "DIVA: A reliable substrate for deep submicron microarchitecture design," in *Proc. IEEE/ACM Symp. on Microarch. (MICRO)*, 1999, pp. 196–207.

[11] S. Dutta, S. Jha, S. Sankaranarayanan, and A. Tiwari, "Output range analysis for deep feedforward neural networks," in *NASA Formal Methods Symposium*. Springer, 2018, pp. 121–138.

[12] K. D. Kusano, J. M. Scanlon, Y.-H. Chen, T. L. McMurry, R. Chen, T. Gode, and T. Victor, "Comparison of Waymo rider-only crash data to human benchmarks at 7.1 million miles," *Traffic Injury Prevention*, vol. 25, no. sup1, pp. S66–S77, 2024.

[13] G. Guo, J. Goldfeder, A. Ray, T. Dear, and H. Lipson, "DeepCollide: Scalable data-driven high DoF configuration space modeling using implicit neural representations," in *Proc. IEEE Int'l Conf. on Computer Vision (ICCV)*, 2025.

[14] D. Nister, H.-L. Lee, J. Ng, and Y. Wang, "Safety procedure analysis for obstacle avoidance in autonomous vehicles," Aug. 3 2021, uS Patent 11,079,764.

[15] W. T. Floyd-Jones, S. M. Murray, G. D. Konidaris, and D. J. Sorin, "Collision detection useful in motion planning for robotics," Jun. 17 2025, uS Patent 12,330,310.

[16] S. Murray, W. Floyd-Jones, Y. Qi, G. Konidaris, and D. J. Sorin, "The microarchitecture of a real-time robot motion planning accelerator," in *Proc. IEEE/ACM Symp. on Microarch. (MICRO)*, 2016.

[17] D. Shah and T. M. Aamodt, "Collision prediction for robotics accelerators," in *Proc. IEEE/ACM Int'l Symp. on Computer Architecture (ISCA)*, 2024.

[18] J. Zhao, D. Shah, and T. M. Aamodt, "Neural encodings for energy-efficient motion planning," in *Proc. IEEE Int'l Conf. on Robotics and Automation (ICRA)*, 2025, pp. 540–547.

[19] P. Leven and S. Hutchinson, "A framework for real-time path planning in changing environments," *The International Journal of Robotics Research*, vol. 21, no. 12, pp. 999–1030, 2002.

[20] D. J. Sorin, G. Konidaris, W. Floyd-Jones, and S. Murray, "Motion planning for autonomous vehicles and reconfigurable motion planning processor," 2019, patent No. US20190163191A1, Filed June 9, 2017, Issued May 30, 2019.

[21] Y. Yang, X. Chen, and Y. Han, "Dadu-CD: Fast and efficient processing-in-memory accelerator for collision detection," in *Proc. ACM/IEEE Design Automation Conference (DAC)*, 2020.

[22] M. Bakhshalipour, S. B. Ehsani, M. Qadri, D. Guri, M. Likhachev, and P. B. Gibbons, "RACOD: Algorithm/hardware co-design for mobile robot path planning," in *Proc. IEEE/ACM Int'l Symp. on Computer Architecture (ISCA)*, 2022.

[23] Z. Wan, Y. Du, M. Ibrahim, J. Qian, J. Jabbour, Y. Zhao, T. Krishna, A. Raychowdhury, and V. J. Reddi, "ReCA: Integrated acceleration for real-time and efficient cooperative embodied autonomous agents," in *Proc. ACM Conf. on Arch. Support for Prog. Lang. and Op. Sys. (ASPLOS)*, 2025.

[24] L. Huang, Y. Gong, Y. Sui, X. Zang, and B. Yuan, "MOPED: Efficient motion planning engine with flexible dimension support," in *Proc. IEEE Symp. on High-Perf. Computer Architecture (HPCA)*, 2024.

[25] K. Koide, "Robotics conference acceptance rate history," https://staff.aist.go.jp/k.koide/acceptance-rate.html, 2025, accessed: 2026-04-01.

[26] I. Tesla, "Q3 2025 financial results and Q&A webcast," Webcast, Investor Relations, Oct. 2025, audio recording of recent AI5 design disclosures available starting at 35 minutes, 14 seconds here: https://youtu.be/GQ9S7xbkGAY?t=2114.

[27] I. Kuon and J. Rose, "Measuring the gap between FPGAs and ASICs," in *Proc. ACM/SIGDA Int'l Symp. on Field Programmable Gate Arrays*, 2006, pp. 21–30.

[28] N. Jouppi, C. Young, N. Patil, and D. Patterson, "Motivation for and evaluation of the first tensor processing unit," *IEEE Micro*, vol. 38, no. 3, pp. 10–19, 2018.

[29] R. Hameed, W. Qadeer, M. Wachs, O. Azizi, A. Solomatnikov, B. C. Lee, S. Richardson, C. Kozyrakis, and M. Horowitz, "Understanding sources of inefficiency in general-purpose chips," in *Proc. IEEE/ACM Int'l Symp. on Computer Architecture (ISCA)*, 2010, pp. 37–47.

[30] R. Raj, H. Wang, and T. Krishna, "A CPU-centric perspective on agentic AI," *arXiv preprint arXiv:2511.00739*, 2025.

[31] Nvidia Corporation. (2018) Nvidia Turing GPU architecture. [Online]. Available: https://images.nvidia.com/aem-dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf

[32] D. K. Mandarapu, N. James, and M. Kulkarni, "Mochi: Fast & exact collision detection," *arXiv preprint arXiv:2402.14801*, 2024.

[33] H. Min, K. M. Han, and Y. J. Kim, "OctoMap-RT: Fast probabilistic volumetric mapping using ray-tracing GPUs," *IEEE Robotics and Automation Letters*, vol. 8, no. 9, 2023.

[34] S. Sui, L. Sentis, and A. Bylard, "Hardware-accelerated ray tracing for discrete and continuous collision detection on GPUs," in *Proc. IEEE Int'l Conf. on Robotics and Automation (ICRA)*, 2025.

[35] D. Ha, L. Liu, Y. H. Chou, S. Go, W. W. Ro, H.-W. Tseng, and T. M. Aamodt, "Generalizing ray tracing accelerators for tree traversals on GPUs," in *Proc. IEEE/ACM Symp. on Microarch. (MICRO)*, 2024.

[36] M. Bakhshalipour and P. B. Gibbons, "Agents of autonomy: A systematic study of robotics on modern hardware," *Proc. ACM on Measurement and Analysis of Computing Systems (POMACS)*, vol. 7, no. 3, pp. 1–31, 2023.

[37] M. Likhachev, D. Ferguson, G. J. Gordon, A. Stentz, and S. Thrun, "Anytime dynamic A*: An anytime, replanning algorithm," *ICAPS*, 2005.

[38] M. Likhachev, G. J. Gordon, and S. Thrun, "ARA*: Anytime A* with provable bounds on suboptimality," *ICAPS*, 2005.

[39] S. M. LaValle, "Rapidly-exploring random tress : a new tool for path planning," *The annual research report*, 1998.

[40] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *The International Journal of Robotics Research*, 2011.

[41] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Batch informed trees (bit*): Sampling-based optimal planning via the heuristically guided search of implicit random geometrics graphs," *Proc. IEEE Int'l Conf. on Robotics and Automation (ICRA)*, 2015.

[42] M. Dalal, J. Yang, R. Mendonca, Y. Khaky, R. Salakhutdinov, and D. Pathak, "Neural MP: A generalist neural motion planner," *Proc. Conf. on Robot Learning (CoRL)*, 2024.

[43] Z. Huang, H. Chen, J. Pohovey, and K. Driggs-Campbell, "Neural informed RRT*: Learning-based path planning with point cloud state representations under admissible ellipsoidal constraints," in *Proc. IEEE Int'l Conf. on Robotics and Automation (ICRA)*, 2024, pp. 8742–8748.

[44] M. Yu, C. Yu, M.-M. Naddaf-Sh, D. Upadhyay, S. Gao, and C. Fan, "Efficient motion planning for manipulators with control barrier function-induced neural controller," in *Proc. IEEE Int'l Conf. on Robotics and Automation (ICRA)*, 2024, pp. 14 348–14 355.

[45] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "PointNet++: Deep hierarchical feature learning on point sets in a metric space," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[46] X. Ma, S. Patidar, I. Haughton, and S. James, "Hierarchical diffusion policy for kinematics-aware multi-task robotic manipulation," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 18 081–18 090.

[47] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The Int'l Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.

[48] Z. Deng, L. Wang, W. Han, R. Ranjan, and A. Zomaya, "G-ML-Octree: An update-efficient index structure for simulating 3D moving objects across GPUs," *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, vol. 29, no. 5, pp. 1075–1088, 2017.

[49] P. Chen, M. Li, Z. Wan, Y.-S. Hsiao, M. Yu, V. J. Reddi, and Z. Liu, "OctoCache: Caching voxels for accelerating 3D occupancy mapping in autonomous systems," in *Proc. ACM Conf. on Arch. Support for Prog. Lang. and Op. Sys. (ASPLOS)*, 2025.

[50] S. Durvasula, R. Kiguru, S. Mathur, J. Xu, J. Lin, and N. Vijaykumar, "VoxelCache: Accelerating online mapping in robotics and 3D reconstruction tasks," in *Proc. IEEE/ACM Conf. on Par. Arch. and Comp. Tech. (PACT)*, 2022.

[51] S. Gottschalk, M. C. Lin, and D. Manocha, "OBBTree: a hierarchical structure for rapid interference detection," in *Proc. Int'l Conf. on Computer Graphics and Interactive Techniques (SIGGRAPH)*, 1996.

[52] D. Shah, N. Yang, and T. M. Aamodt, "Energy-efficient realtime motion planning," in *Proc. IEEE/ACM Int'l Symp. on Computer Architecture (ISCA)*, 2023.

[53] Intel Corporation. (2022) Introduction to the Xe-HPG architecture. [Online]. Available: https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-the-xe-hpg-architecture.html

[54] Advanced Micro Devices Inc. (2022) AMD RDNA architecture. [Online]. Available: https://www.amd.com/en/technologies/rdna

[55] Imagination Technologies. (2022) Rays your game: Introduction to the PowerVR photon architecture. [Online]. Available: https://www.imaginationtech.com/resources/rays-your-game-en/

[56] Apple. (2023) Apple unveils M3, M3 Pro, and M3 Max, the most advanced chips for a personal computer. [Online]. Available: https://www.apple.com/newsroom/2023/10/apple-unveils-m3-m3-pro-and-m3-max-the-most-advanced-chips-for-a-personal-computer/

[57] J. B. Dennis and D. P. Misunas, "A preliminary architecture for a basic data-flow processor," in *Proc. IEEE/ACM Int'l Symp. on Computer Architecture (ISCA)*, 1974, pp. 126–132.

[58] I. Evangelou, G. Papaioannou, K. Vardis, and A. A. Vasilakis, "Fast radius search exploiting ray-tracing frameworks," *Journal of Computer Graphics Techniques (JCGT)*, vol. 10, no. 1, 2021.

[59] Y. Zhu, "RTNN: accelerating neighbor search using hardware ray tracing," in *Proc. ACM Symp. on Prin. and Prac. of Par. Prog. (PPoPP)*, 2022, pp. 76–89.

[60] H. Zhang, Y. Zhang, and H.-W. Tseng, "RTSpMSpM: Harnessing ray tracing for efficient sparse matrix computations," in *Proc. IEEE/ACM Int'l Symp. on Computer Architecture (ISCA)*, 2025.

[61] J. Henneberg and F. Schuhknecht, "RTIndeX: Exploiting hardware-accelerated GPU raytracing for database indexing," *Proc. of the VLDB Endowment*, vol. 16, no. 13, pp. 4268–4281, 2023.

[62] V. Nagarajan and M. Kulkarni, "RT-DBSCAN: Accelerating DBSCAN using ray tracing hardware," in *Proc. IEEE Int'l Parallel and Distributed Processing Symp. (IPDPS)*, 2023, pp. 963–973.

[63] H. Min, K. M. Han, and Y. J. Kim, "Accelerating probabilistic volumetric mapping using ray-tracing graphics hardware," in *Proc. IEEE Int'l Conf. on Robotics and Automation (ICRA)*, 2021.

[64] Y. H. Chou, T. Nowicki, and T. M. Aamodt, "Treelet prefetching for ray tracing," in *Proc. IEEE/ACM Symp. on Microarch. (MICRO)*, 2023.

[65] Y. S. Tozlu and H. Zhou, "CoopRT: Accelerating BVH traversal for ray tracing via cooperative threads," in *Proc. IEEE/ACM Int'l Symp. on Computer Architecture (ISCA)*, 2025.

[66] L. Liu, W. Chang, F. Demoullin, Y. H. Chou, M. Saed, D. Pankratz, T. Nowicki, and T. M. Aamodt, "Intersection prediction for accelerated GPU ray tracing," in *Proc. IEEE/ACM Symp. on Microarch. (MICRO)*, 2021, pp. 709–723.

[67] Arvind and R. S. Nikhil, "Executing a program on the mit tagged-token dataflow architecture," in *Int'l Conf. on Parallel Architectures and Languages Europe*, 1987.

[68] M. Saed, Y. H. Chou, L. Liu, T. Nowicki, and T. M. Aamodt, "Vulkan-Sim: A GPU architecture simulator for ray tracing," in *Proc. IEEE/ACM Symp. on Microarch. (MICRO)*, 2022, pp. 263–281.

[69] Y. H. Chou and T. M. Aamodt, "Treelet accelerated ray tracing on GPUs," in *Proc. ACM Conf. on Arch. Support for Prog. Lang. and Op. Sys. (ASPLOS)*, 2025, pp. 1334–1347.

[70] Y.-C. Huang, C.-P. Yang, and T. T. Yeh, "AQB8: Energy-efficient ray tracing accelerator through multi-level quantization," in *Proc. IEEE/ACM Int'l Symp. on Computer Architecture (ISCA)*, 2025.

[71] Y. Feng, Y. Li, J. Lee, W. W. Ro, and H. Jeon, "Heliostat: Harnessing ray tracing accelerators for page table walks," in *Proc. IEEE/ACM Int'l Symp. on Computer Architecture (ISCA)*, 2025.

[72] M. Saed, P. J. Nair, and T. M. Aamodt, "RayN: Ray tracing acceleration with near-memory computing," in *Proc. IEEE/ACM Symp. on Microarch. (MICRO)*, 2025.

[73] H. Wang, W. Liu, K. Chen, Q. Sun, and S. Q. Zhang, "Process only where you look: Hardware and algorithm co-optimization for efficient gaze-tracked foveated rendering in virtual reality," in *Proc. IEEE/ACM Int'l Symp. on Computer Architecture (ISCA)*, 2025.

[74] D. Nikiforov, S. C. Dong, C. L. Zhang, S. Kim, B. Nikolic, and Y. S. Shao, "Rosé: A hardware-software co-simulation infrastructure enabling pre-silicon full-stack robotics soc evaluation," in *Proc. IEEE/ACM Int'l Symp. on Computer Architecture (ISCA)*, 2023.

[75] V. Kandiah, S. Peverelle, M. Khairy, J. Pan, A. Manjunath, T. G. Rogers, T. M. Aamodt, and N. Hardavellas, "AccelWattch: A Power Modeling Framework for Modern GPUs," in *Proc. IEEE/ACM Symp. on Microarch. (MICRO)*, 2021, pp. 738–753.

[76] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avižienis, J. Wawrzynek, and K. Asanović, "Chisel: constructing hardware in a scala embedded language," in *Proc. ACM/IEEE Design Automation Conference (DAC)*, 2012.

[77] J. R. Hauser, "Berkeley HardFloat: Verilog Modules for Floating-Point Arithmetic," https://www.jhauser.us/arithmetic/HardFloat.html, 2019, release 1, accessed: 2026-03-20.

[78] J. E. Stine, I. Castellanos, M. Wood, J. Henson, F. Love, W. R. Davis, P. D. Franzon, M. Bucher, S. Basavarajaiah, J. Oh *et al.*, "FreePDK: An open-source variation-aware design kit," in *IEEE Int'l Conf. on Microelectronic Systems Education (MSE)*, 2007.

[79] NVIDIA, "Physical AI Autonomous Vehicles Dataset," https://huggingface.co/datasets/nvidia/PhysicalAI-Autonomous-Vehicles, 2025, accessed: 2026-03-20.

[80] I. Wald, S. Woop, C. Benthin, G. S. Johnson, and M. Ernst, "Embree: a kernel framework for efficient CPU ray tracing," *ACM Transactions on Graphics (TOG)*, vol. 33, no. 4, 2014.

[81] Khronos Group. (2025) Vulkan. [Online]. Available: https://www.vulkan.org/

[82] B. Sundaralingam, S. K. S. Hari, A. F. C. Garrett, K. V. W. V. B. Alexander, M. H. Oleynikova, A. H. F. R. N. Ratliff, and D. Fox, "CuRobo: Parallelized collision-free minimum-jerk robot motion generation," *arXiv preprint arXiv:2310.17274*, 2023.

[83] K. Sugiura and H. Matsutani, "An integrated FPGA accelerator for deep learning-based 2D/3D path planning," *IEEE Transactions on Computers*, vol. 73, no. 6, pp. 1442–1456, 2024.

[84] R. C. Hoetzlein, "Fast fixed-radius nearest neighbors: interactive million-particle fluids," in *GPU Technology Conference*, vol. 18, 2014, p. 2.

[85] S. Liu, Y. Zhou, J. Yang, S. Gupta, and S. Wang, "ContactGen: Generative contact modeling for grasp generation," in *Proc. IEEE Int'l Conf. on Computer Vision (ICCV)*, 2023, pp. 20 609–20 620.

[86] S. Chen, R. Garcia, I. Laptev, and C. Schmid, "Sugar: Pre-training 3D visual representations for robotics," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 18 049–18 060.

[87] R. Xu, H. Lin, W. Jeon, H. Feng, Y. Zou, L. Sun, J. Gorman, K. Tolstaya, S. Tang, B. White *et al.*, "WOD-E2E: Waymo open dataset for end-to-end driving in challenging long-tail scenarios," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2026.

[88] L. Liu, M. Saed, Y. H. Chou, D. Grigoryan, T. Nowicki, and T. M. Aamodt, "LumiBench: A benchmark suite for hardware ray tracing," in *Proc. IEEE Symp. on Workload Characterization (IISWC)*, 2023.

[89] F. Shen, A. Barnes, A. Nallathambi, and T. G. Rogers, "RayFlex: An open-source RTL implementation of the hardware ray tracer datapath," in *Proc. IEEE Symp. on Perf. Analysis of Systems and Software (ISPASS)*, 2025.

[90] F. Chen, R. Ying, J. Xue, F. Wen, and P. Liu, "ParallelNN: A parallel octree-based nearest neighbor search accelerator for 3D point clouds," in *Proc. IEEE Symp. on High-Perf. Computer Architecture (HPCA)*, 2023, pp. 403–414.

[91] J. Sacks, D. Mahajan, R. C. Lawson, and H. Esmaeilzadeh, "RoboX: An end-to-end solution to accelerate autonomous control in robotics," in *Proc. IEEE/ACM Int'l Symp. on Computer Architecture (ISCA)*, 2018.

[92] Y. Feng, B. Tian, T. Xu, P. Whatmough, and Y. Zhu, "Mesorasi: Architecture support for point cloud analytics via delayed aggregation," in *Proc. IEEE/ACM Symp. on Microarch. (MICRO)*, 2020.

[93] S. Li, Y. Zhao, C. Li, B. Guo, J. Zhang, W. Zhu, Z. Ye, C. Wan, and Y. C. Lin, "Fusion-3D: Integrated acceleration for instant 3D reconstruction and real-time rendering," in *Proc. IEEE/ACM Symp. on Microarch. (MICRO)*, 2024.

[94] Y. Huang, Y. Hao, B. Yu, F. Yan, Y. Yang, F. Min, Y. Han, L. Ma, S. Liu, Q. Liu, and Y. Gan, "Dadu-Corki: Algorithm-architecture co-design for embodied AI-powered robotic manipulation," in *Proc. IEEE/ACM Int'l Symp. on Computer Architecture (ISCA)*, 2025.

[95] J. Ting, M. Kim, J. Zhu, H. Sheng, and Z. Zhang, "HiPER: Hierarchically-composed processing for efficient robot learning-based control," in *Proc. IEEE/ACM Int'l Symp. on Computer Architecture (ISCA)*, 2025.

[96] M. Bakhshalipour and P. B. Gibbons, "Tartan: Microarchitecting a robotic processor," in *Proc. IEEE/ACM Int'l Symp. on Computer Architecture (ISCA)*, 2024.

[97] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, U. Malla, D. Manjunath, I. Mordatch, O. Nachum, C. Parada, J. Peralta, E. Perez, K. Pertsch, J. Quiambao, K. Rao, M. Ryoo, G. Salazar, P. Sanketi, K. Sayed, J. Singh, S. Sontakke, A. Stone, C. Tan, H. Tran, V. Vanhoucke, S. Vega, Q. Vuong, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and

B. Zitkovich, "RT-1: Robotics transformer for real-world control at scale," in *arXiv preprint arXiv:2212.06817*, 2022.

[98] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. J. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llontop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu, "Gr00t n1: An open foundation model for generalist humanoid robots," *arXiv preprint arXiv:2503.14734*, 2025.

[99] N. Amato and L. Dale, "Probabilistic roadmap methods are embarrassingly parallel," in *Proc. IEEE Int'l Conf. on Robotics and Automation (ICRA)*, 1999.

[100] R. Gayle, P. Segars, M. Lin, and D. Manocha, "Path planning for deformable robots in complex environments," in *Robotics: Science and Systems*, 2005.

[101] J. Bialkowski, S. Karaman, and E. Frazzoli, "Massively parallelizing the RRT and the RRT," in *Proc. IEEE/RSJ Int'l Conf. on Intelligent Robots and Systems (IROS)*, 2011.

[102] J. T. Kider, M. Henderson, M. Likhachev, and A. Safonova, "High-dimensional planning on the GPU," in *Proc. IEEE Int'l Conf. on Robotics and Automation (ICRA)*, 2010.

[103] J. Pan and D. Manocha, "GPU-based parallel collision detection for fast motion planning," *The International Journal of Robotics Research*, vol. 31, no. 2, pp. 187–200, 2012.

[104] Y. Hao, Y. Gan, B. Yu, Q. Liu, Y. Han, Z. Wan, and S. Liu, "Orianna: An accelerator generation framework for optimization-based robotic applications," in *Proc. ACM Conf. on Arch. Support for Prog. Lang. and Op. Sys. (ASPLOS)*, 2024.

[105] Y. Hao, Y. Gan, B. Yu, Q. Liu, S.-S. Liu, and Y. Zhu, "Blitzcrank: Factor graph accelerator for motion planning," in *Proc. ACM/IEEE Design Automation Conference (DAC)*, 2023.

[106] C. Lienen, M. Platzner, and B. Rinner, "ReconROS: Flexible hardware acceleration for ROS2 applications," in *Int'l Conf. on Field-Programmable Technology (ICFPT)*, 2020.

[107] N. Atay and B. Bayazit, "A motion planning processor on reconfigurable hardware," in *Proc. IEEE Int'l Conf. on Robotics and Automation (ICRA)*, 2006.

[108] X. Shi, L. Cao, D. Wang, L. Liu, G. You, S. Liu, and C. Wang, "HERO: Accelerating Autonomous Robotic Tasks with FPGA," 2018.

[109] Y. Zhang, X. Niu, H. Tian, Y. Zhang, B. Yu, S. Liu, and S. Huang, "A sparsity-aware autonomous path planning accelerator with HW/SW co-design and multi-level dataflow optimization," *ACM Transactions on Architecture and Code Optimization (TACO)*, 2025.

[110] S. Murray, W. Floyd-jones, G. Konidaris, and D. J. Sorin, "A Programmable Architecture for Robot Motion Planning Acceleration," in *Int'l Conf. on Application-specific Systems, Architectures and Processors*, 2019.

[111] S. Lian, Y. Han, X. Chen, Y. Wang, and H. Xiao, "Dadu-P: A scalable accelerator for robot motion planning in a dynamic environment," in *Proc. ACM/IEEE Design Automation Conference (DAC)*, 2018.

[112] Z. Li, T. Ren, X. He, and C. Liu, "RED: A systematic real-time scheduling approach for robotic environmental dynamics," in *IEEE Real-Time Systems Symposium (RTSS)*, 2023, pp. 210–223.

[113] L. Liu and T. M. Aamodt, "ReFrame: Layer caching for accelerated inference in real-time rendering," in *Proc. of Int'l Conf. on Machine Learning (ICML)*, 2025.

[114] J. Elseberg, D. Borrmann, and A. Nüchter, "Efficient processing of large 3D point clouds," in *International Symposium on Information, Communication and Automation Technologies*, 2011.

[115] ——, "One billion points in the cloud - an octree for efficient processing of 3D laser scans," *ISPRS Journal of Photogrammetry and Remote Sensing*, 2013.

[116] Y.-T. Su, J. Bethel, and S. Hu, "Octree-based segmentation for terrestrial LiDAR point cloud data in industrial applications," *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 113, pp. 59–74, 2016.

[117] H. Xu, L. Cheng, M. Li, Y. Chen, and L. Zhong, "Using octrees to detect changes to buildings and trees in the urban environment from airborne LiDAR data," *Remote Sensing*, 2015.

[118] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," *Autonomous robots*, vol. 34, no. 3, pp. 189–206, 2013.

[119] J. Zhu, H. Li, Z. Wang, S. Wang, and T. Zhang, "i-Octree: A fast, lightweight, and dynamic octree for proximity search," in *Proc. IEEE Int'l Conf. on Robotics and Automation (ICRA)*, 2024.

[120] C. Lauterbach, M. Garland, S. Sengupta, D. Luebke, and D. Manocha, "Fast BVH construction on GPUs," in *Computer Graphics Forum*, 2009.

[121] D. Meister and J. Bittner, "Parallel locally-ordered clustering for bounding volume hierarchy construction," *IEEE Transactions on Visualization and Computer Graphics (TVCG)*, vol. 24, no. 3, 2017.

[122] C. Benthin, D. Meister, J. Barczak, R. Mehalwal, J. Tsakok, and A. Kensler, "H-PLOC: Hierarchical parallel locally-ordered clustering for bounding volume hierarchy construction," *Proc. ACM Conf. on High Performance Graphics (HPG)*, 2024.

[123] C. Benthin, R. Drabinski, L. Tessari, and A. Dittebrandt, "PLOC++ parallel locally-ordered clustering for bounding volume hierarchy construction revisited," *Proc. ACM Conf. on High Performance Graphics (HPG)*, 2022.

[124] H. Ylitie, T. Karras, and S. Laine, "Efficient incoherent ray traversal on GPUs through compressed wide BVHs," in *Proc. ACM Conf. on High Performance Graphics (HPG)*, 2017.